

NHẬP MÔN

Golang & Apache Kafka

Xây dựng hệ thống xử lý message thời gian thực

Tác giả: Giảng viên Bộ môn Lập trình
Phiên bản: 1.0.0
Ngày: Tháng 4, 2025
Ngôn ngữ: Go 1.22+
Kafka: apache/kafka:3.8.1

Tài liệu nội bộ — Dành cho sinh viên năm 2, 3 chuyên ngành Công nghệ phần mềm

Mục Lục

Mục Lục	2
Chương 1: Giới Thiệu Tổng Quan	4
1.1 Go là gì?	4
1.1.1 Đặc điểm nổi bật của Go	4
1.1.2 Tại sao dùng Go cho hệ thống phân tán?	4
1.2 Apache Kafka là gì?	4
1.2.1 Vấn đề Kafka giải quyết	5
Chương 2: Chuẩn Bị Môi Trường	6
2.1 Cài đặt Go	6
2.1.1 Ubuntu / macOS	6
2.1.2 Kiểm tra cài đặt Go	6
2.2 Cài đặt Docker	6
2.3 Khởi động Kafka với Docker Compose	7
2.3.1 Kiểm tra Kafka đang chạy	7
2.4 Các lệnh Docker Compose cần nhớ	7
Chương 3: Các Khái Niệm Cơ Bản về Kafka	9
3.1 Kiến trúc tổng quan	9
3.2 Giải thích từng khái niệm	9
3.2.1 Message (Tin nhắn)	9
3.2.2 Topic (Chủ đề)	9
3.2.3 Partition (Phân vùng)	9
3.2.4 Offset (Vị trí đọc)	10
3.2.5 Broker	10
3.2.6 Producer (Bên gửi)	10
3.2.7 Consumer và Consumer Group	10
Chương 4: Thiết Kế Cấu Trúc Thư Mục Project	12
4.1 Cấu trúc thư mục đề xuất	12
4.2 Giải thích vai trò từng thư mục	12
4.3 Khởi tạo project	12
Chương 5: Thực Hành — Kết Nối Go với Kafka	14
5.1 Cấu hình kết nối — config/config.go	14
5.2 Model dữ liệu — models/order.go	14
5.3 Lớp Kafka — internal/kafka/producer.go	15
5.4 Lớp Kafka — internal/kafka/consumer.go	16
5.5 Producer — cmd/producer/main.go	18
5.6 Consumer — cmd/consumer/main.go	19
5.7 Chạy thử và kiểm tra kết quả	21
Bước 1: Đảm bảo Kafka đang chạy	21

Bước 2: Chạy Consumer (Terminal A)	21
Bước 3: Chạy Producer (Terminal B)	21
Bước 4: Quan sát Consumer nhận message (Terminal A)	22
Bước 5: Kiểm tra qua Redpanda Console	22
Chương 6: Lỗi Thường Gặp và Cách Xử Lý	23
6.1 Lỗi kết nối Kafka	23
6.2 Lỗi topic chưa tồn tại	23
6.3 Lỗi biên dịch Go	23
6.4 Consumer không nhận được message	24
6.5 Lỗi context deadline exceeded	24
6.6 Bảng tổng hợp lỗi nhanh	24
Chương 7: Tổng Kết	26
7.1 Những gì bạn đã học được	26
7.2 Bảng ôn tập khái niệm	26
7.3 Lộ trình học tiếp theo	26
7.3.1 Chủ đề nâng cao về Go	26
7.3.2 Chủ đề nâng cao về Kafka	27
7.3.3 Tài nguyên học thêm	27

Chương 1: Giới Thiệu Tổng Quan

Trước khi bắt tay vào code, chúng ta cần hiểu rõ mình đang dùng công cụ gì và tại sao lại dùng nó. Đây không phải lý thuyết suông — hiểu đúng bản chất sẽ giúp bạn debug nhanh hơn và đưa ra quyết định kỹ thuật tốt hơn về sau.

1.1 Go là gì?

Go (hay còn gọi là Golang) là ngôn ngữ lập trình được Google phát triển và công bố vào năm 2009. Nhóm kỹ sư đứng sau Go — gồm Robert Griesemer, Rob Pike, và Ken Thompson — đều là những tên tuổi huyền thoại trong lịch sử khoa học máy tính.

Ý tưởng khi tạo ra Go rất đơn giản: Google cần một ngôn ngữ vừa nhanh như C++, vừa dễ viết như Python, và đặc biệt phải xử lý tốt việc lập trình đa luồng (concurrency) — thứ mà C++ và Java làm rất phức tạp, còn Python lại làm quá chậm.

1.1.1 Đặc điểm nổi bật của Go

- Biên dịch sang mã máy (compiled): Go biên dịch trực tiếp ra file thực thi nhị phân, không cần máy ảo (JVM) hay trình thông dịch. Điều này khiến chương trình Go khởi động gần như tức thì.
- Quản lý bộ nhớ tự động (Garbage Collection): Bạn không cần tự giải phóng bộ nhớ như C/C++, nhưng hiệu năng vẫn cao hơn nhiều so với Java hay Python.
- Concurrency là công dân hạng nhất: Go có goroutine và channel được tích hợp sẵn trong ngôn ngữ. Chạy 10.000 goroutine đồng thời là chuyện bình thường — thứ mà thread của Java/Python không thể làm được.
- Cú pháp đơn giản, học nhanh: Go không có class, không có kế thừa, không có exception kiểu truyền thống. Chỉ có struct, interface, và error trả về tường minh.
- Toolchain mạnh mẽ: go build, go test, go fmt, go vet — tất cả đều có sẵn, không cần cài thêm gì.

1.1.2 Tại sao dùng Go cho hệ thống phân tán?

Hệ thống phân tán (distributed system) là những hệ thống có nhiều máy chủ cùng làm việc với nhau — ví dụ hệ thống thanh toán của ngân hàng, feed của mạng xã hội, hay hệ thống đặt vé máy bay. Những hệ thống này có yêu cầu rất khắt khe:

- Xử lý hàng nghìn request mỗi giây
- Độ trễ thấp (< 10ms)
- Không được sập dù một node bị lỗi
- Dễ scale ngang (thêm máy chủ)

Go đáp ứng tất cả những yêu cầu trên. Không ngạc nhiên khi Docker, Kubernetes, Prometheus, và hầu hết cơ sở hạ tầng cloud-native hiện đại đều được viết bằng Go.

Fun fact

Kafka, Elasticsearch, và Cassandra được viết bằng Java.
Kubernetes, Docker, Prometheus, etcd được viết bằng Go.
Trong thực tế, Go và Java/Scala thường sát cánh nhau trong hệ thống lớn.

1.2 Apache Kafka là gì?

Apache Kafka là một nền tảng stream processing (xử lý luồng dữ liệu) phân tán, được LinkedIn phát triển vào năm 2011 và sau đó mã nguồn mở hóa thông qua Apache Foundation.

Để hiểu Kafka, hãy nghĩ đến một ví dụ thực tế: Bạn đang xây dựng hệ thống thương mại điện tử. Khi một đơn hàng được đặt, hàng chục việc cần xảy ra đồng thời:

- Gửi email xác nhận cho khách hàng
- Cập nhật số lượng tồn kho
- Thông báo cho kho hàng chuẩn bị gói hàng
- Cập nhật dashboard thống kê
- Tính hoa hồng cho affiliate

Nếu làm theo cách truyền thống (gọi trực tiếp từng service), code sẽ rất rối, và nếu một service bị lỗi, toàn bộ quy trình bị ảnh hưởng. Kafka giải quyết bài toán này bằng mô hình publish-subscribe.

1.2.1 Vấn đề Kafka giải quyết

Vấn đề	Không có Kafka	Có Kafka
Kết nối service	Tight coupling — service A phải biết B, C, D	Loose coupling — chỉ cần biết topic
Xử lý lỗi	Lỗi 1 service làm hỏng cả chuỗi	Message được lưu lại, retry tự động
Scale	Phải scale đồng bộ cả hệ thống	Scale từng consumer độc lập
Tốc độ	Bị giới hạn bởi service chậm nhất	Producer không bị block
Audit log	Phải tự làm, dễ bỏ sót	Message được lưu tự động theo retention

Tóm tắt chương

Go là ngôn ngữ biên dịch, nhanh, concurrency tốt, cú pháp đơn giản.
 Kafka là message broker phân tán, giải quyết giao tiếp giữa các service.
 Kết hợp Go + Kafka = nền tảng lý tưởng cho hệ thống backend hiện đại.
 Cả hai đều có cộng đồng lớn, tài liệu đầy đủ, và được dùng rộng rãi trong thực tế.

Chương 2: Chuẩn Bị Môi Trường

Chương này hướng dẫn bạn cài đặt tất cả những gì cần thiết để bắt đầu. Hãy làm từng bước và xác nhận mỗi bước thành công trước khi đi tiếp.

2.1 Cài đặt Go

2.1.1 Ubuntu / macOS

Truy cập <https://go.dev/dl/> để tải phiên bản mới nhất. Tại thời điểm viết tài liệu, phiên bản ổn định là Go 1.22.

```
# Ubuntu: Tải và cài Go 1.22
wget https://go.dev/dl/go1.22.0.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.22.0.linux-amd64.tar.gz

# Thêm Go vào PATH — mở file ~/.bashrc (Ubuntu) hoặc ~/.zshrc (macOS)
echo "export PATH=$PATH:/usr/local/go/bin" >> ~/.bashrc
source ~/.bashrc

# macOS (dùng Homebrew — cách dễ nhất)
brew install go
```

Windows

Tải file .msi từ <https://go.dev/dl/> và chạy installer.
Installer sẽ tự động thêm Go vào PATH.
Mở Command Prompt mới và kiểm tra: `go version`

2.1.2 Kiểm tra cài đặt Go

```
# Kiểm tra phiên bản Go
go version

# Kết quả mong đợi:
# go version go1.22.0 linux/amd64

# Kiểm tra biến môi trường Go
go env GOPATH
go env GOROOT
```

2.2 Cài đặt Docker

Docker cho phép chạy Kafka mà không cần cài đặt Java hay cấu hình phức tạp. Chúng ta chỉ cần một lệnh để khởi động toàn bộ hệ thống Kafka.

```
# Ubuntu: Cài Docker chính thức
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker # Áp dụng quyền ngay mà không cần logout

# macOS: Cài Docker Desktop
# Tải từ https://www.docker.com/products/docker-desktop/
```

```
# Kiểm tra Docker hoạt động
docker version
docker compose version
```

2.3 Khởi động Kafka với Docker Compose

File docker-compose.yml đã được cung cấp sẵn. Hệ thống sẽ bao gồm 2 service: Kafka broker (KRaft mode, không cần Zookeeper) và Redpanda Console (giao diện web quản lý).

```
# Tạo thư mục project
mkdir -p ~/kafka-go-demo
cd ~/kafka-go-demo

# Sao chép file docker-compose.yml vào thư mục này
# (đã được cung cấp trong tài liệu hướng dẫn)

# Khởi động Kafka (chạy nền với flag -d)
docker compose up -d

# Xem trạng thái các container
docker compose ps

# Kết quả mong đợi (sau ~30 giây):
# NAME                                STATUS          PORTS
# my-kafka                            Up (healthy)    0.0.0.0:9092->9092/tcp
# redpanda-console                    Up (healthy)    0.0.0.0:9091->8080/tcp
```

Lưu ý quan trọng

Lần đầu chạy, Docker sẽ tải image từ internet (~500MB). Hãy đợi.
Kafka cần ~30 giây để khởi động hoàn toàn. Container sẽ hiển thị "(healthy)" khi sẵn sàng.
Nếu healthcheck bị stuck, hãy xem log: `docker compose logs kafka`

2.3.1 Kiểm tra Kafka đang chạy

```
# Cách 1: Kiểm tra qua Redpanda Console (giao diện đồ họa)
# Mở trình duyệt và truy cập: http://localhost:9091

# Cách 2: Kiểm tra qua lệnh trong container
docker exec my-kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 --list

# Cách 3: Kiểm tra cổng đang lắng nghe
nc -zv localhost 9092
# Kết quả: Connection to localhost 9092 port [tcp/*] succeeded!
```

2.4 Các lệnh Docker Compose cần nhớ

Lệnh	Ý nghĩa
<code>docker compose up -d</code>	Khởi động tất cả service ở chế độ nền
<code>docker compose down</code>	Dừng và xóa container (giữ lại dữ liệu)

<code>docker compose logs -f kafka</code>	Xem log realtime của Kafka
<code>docker compose ps</code>	Xem trạng thái các container
<code>docker compose restart kafka</code>	Khởi động lại Kafka

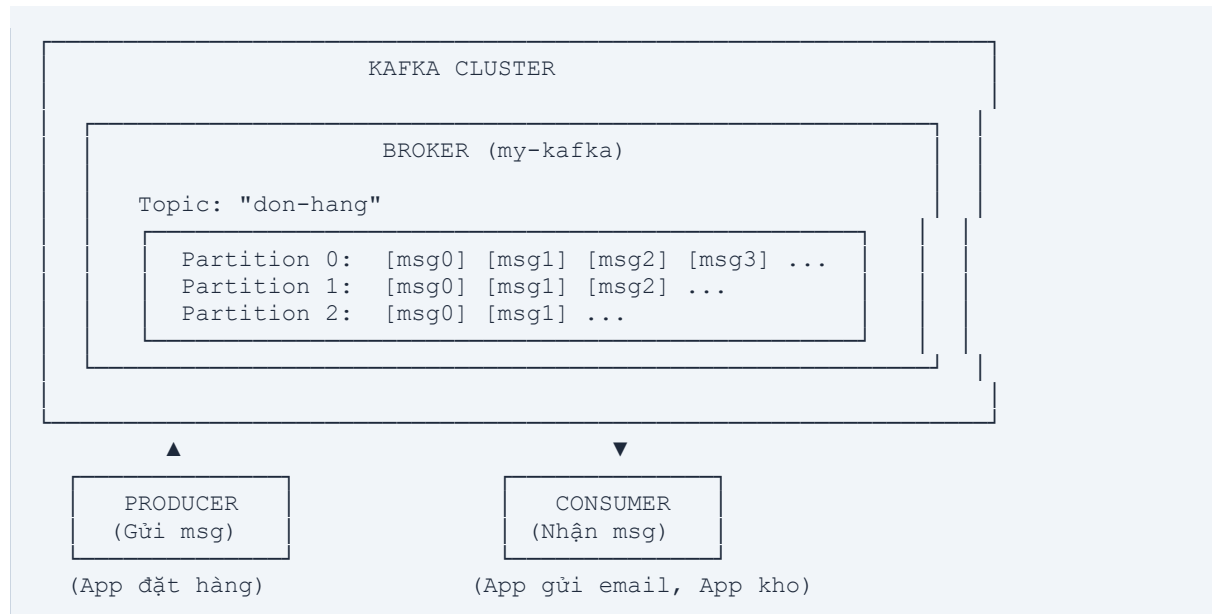
Tóm tắt chương

Cài Go 1.22+ và xác nhận bằng lệnh `go version`.
 Cài Docker (bao gồm Docker Compose plugin).
 Chạy `docker compose up -d` để khởi động Kafka.
 Kiểm tra Kafka tại <http://localhost:9091> (Redpanda Console).
 Kafka lắng nghe ở cổng 9092, Redpanda Console ở cổng 9091.

Chương 3: Các Khái Niệm Cơ Bản về Kafka

Trước khi code, chúng ta cần nắm chắc 7 khái niệm cốt lõi của Kafka. Đừng bỏ qua chương này — hiểu sai khái niệm sẽ dẫn đến bug rất khó tìm về sau.

3.1 Kiến trúc tổng quan



3.2 Giải thích từng khái niệm

3.2.1 Message (Tin nhắn)

Message là đơn vị dữ liệu cơ bản trong Kafka. Mỗi message gồm 3 phần chính:

- **Key:** Định danh tùy chọn, dùng để quyết định message sẽ đi vào partition nào (cùng key luôn vào cùng partition). Ví dụ: mã đơn hàng "ORD-001".
- **Value:** Nội dung thực sự của message. Thường là JSON, Protobuf, hay Avro. Ví dụ: `{ "order_id": "ORD-001", "amount": 150000, "user_id": "U-42" }`.
- **Timestamp:** Thời điểm message được tạo, gắn tự động bởi Kafka.
- **Headers:** Metadata tùy chọn dạng key-value, dùng cho routing hay tracing.

3.2.2 Topic (Chủ đề)

Topic là "kênh" để phân loại message. Hãy nghĩ Topic như một cái hộp thư — Producer bỏ thư vào, Consumer lấy thư ra. Mỗi ứng dụng thường có nhiều topic, ví dụ:

- "don-hang-moi" — các đơn hàng vừa được đặt
- "thanh-toan-thanh-cong" — xác nhận thanh toán
- "ton-kho-thay-doi" — cập nhật số lượng tồn kho

Topic không tự động xóa message sau khi Consumer đọc xong. Message được giữ lại theo cấu hình retention (mặc định 7 ngày). Đây là điểm khác biệt lớn so với hàng đợi truyền thống (RabbitMQ).

3.2.3 Partition (Phân vùng)

Mỗi Topic được chia thành nhiều Partition. Partition là nơi message được lưu trữ theo thứ tự tuyến tính (append-only log). Đây là cơ chế cho phép Kafka scale:

- Các Partition có thể nằm trên các máy chủ khác nhau — giúp phân tải.
- Nhiều Consumer có thể đọc từ các Partition khác nhau song song.
- Thứ tự message được đảm bảo trong mỗi Partition, nhưng KHÔNG được đảm bảo xuyên Partition.

Ví dụ: Topic "don-hang" với 3 Partition

```
Partition 0: [ORD-001] [ORD-004] [ORD-007] [ORD-010] ...
Partition 1: [ORD-002] [ORD-005] [ORD-008] [ORD-011] ...
Partition 2: [ORD-003] [ORD-006] [ORD-009] [ORD-012] ...
                                     ↑
                               Offset tăng dần
```

3.2.4 Offset (Vị trí đọc)

Offset là số thứ tự của message trong một Partition. Message đầu tiên có offset = 0, tiếp theo là 1, 2, 3... Offset không bao giờ giảm và không bao giờ tái sử dụng.

Consumer lưu lại offset của message cuối cùng đã xử lý. Đây gọi là "commit offset". Khi Consumer restart, nó biết phải đọc tiếp từ đâu nhờ offset đã commit.

⚠ Sai lầm phổ biến

Nhiều bạn nhầm tưởng Kafka xóa message sau khi Consumer đọc xong. Thực ra Kafka chỉ xóa theo retention policy (thời gian hoặc kích thước). Bạn có thể "rewind" về offset cũ để đọc lại message đã xử lý — rất hữu ích khi debug!

3.2.5 Broker

Broker là một server Kafka. Trong môi trường thực tế, một Kafka cluster thường có 3-5 broker để đảm bảo tính sẵn sàng cao (HA). Trong bài học này, chúng ta chỉ dùng 1 broker (container my-kafka).

Mỗi Broker được định danh bằng một số nguyên duy nhất (broker ID). Trong docker-compose, chúng ta đặt KAFKA_NODE_ID: 1, tức là đây là Broker số 1.

3.2.6 Producer (Bên gửi)

Producer là bất kỳ ứng dụng nào ghi message vào Kafka. Producer không cần biết Consumer là ai — nó chỉ cần biết tên Topic và địa chỉ Broker. Đây chính là sức mạnh của mô hình publish-subscribe.

Producer có thể chỉ định Key để message đi vào một Partition cụ thể (useful khi cần đảm bảo thứ tự), hoặc để Kafka tự quyết định (round-robin).

3.2.7 Consumer và Consumer Group

Consumer là ứng dụng đọc message từ Kafka. Consumer Group là nhóm các Consumer cùng làm một nhiệm vụ. Kafka đảm bảo mỗi Partition chỉ được đọc bởi một Consumer trong cùng một Group tại một thời điểm.

Ví dụ: Topic "don-hang" có 3 Partition, Consumer Group "email-service"

Partition 0 → Consumer 1 (instance 1 của email-service)

```
Partition 1 → Consumer 2 (instance 2 của email-service)
Partition 2 → Consumer 3 (instance 3 của email-service)
```

→ Scale email-service từ 1 lên 3 instance = tăng tốc xử lý gấp 3 lần!

Nếu có 4 Consumer cho 3 Partition:

```
Partition 0 → Consumer 1
Partition 1 → Consumer 2
Partition 2 → Consumer 3
(idle)      → Consumer 4 ← đây là Consumer dự phòng, không làm gì
```

Tóm tắt chương

Message = đơn vị dữ liệu (key + value + timestamp).

Topic = kênh phân loại message, không tự xóa sau khi đọc.

Partition = phân vùng trong Topic, đảm bảo thứ tự nội bộ.

Offset = số thứ tự message trong Partition, Consumer lưu lại để biết đọc tiếp từ đâu.

Broker = server Kafka chứa và phục vụ message.

Producer = ghi message, Consumer = đọc message.

Consumer Group = nhóm Consumer cùng làm việc, mỗi Partition chỉ do 1 Consumer đọc.

Chương 4: Thiết Kế Cấu Trúc Thư Mục Project

Cấu trúc thư mục tốt là nền móng của dự án tốt. Đặc biệt trong Go, cộng đồng có những quy ước (convention) rất rõ ràng mà bạn nên tuân theo ngay từ đầu.

4.1 Cấu trúc thư mục đề xuất

```
kafka-go-demo/
├── docker-compose.yml      # Cấu hình Kafka + Redpanda Console
├── go.mod                  # Module file — khai báo tên module và
├── dependencies             # Checksum của dependencies — KHÔNG sửa tay
├── README.md               # Mô tả dự án
├── cmd/                    # Điểm vào (entry point) của các chương trình
│   ├── producer/          # Chương trình Producer
│   │   └── main.go
│   └── consumer/          # Chương trình Consumer
│       └── main.go
├── internal/               # Code nội bộ, KHÔNG xuất ra ngoài module
│   └── kafka/              # Logic kết nối và thao tác với Kafka
│       ├── producer.go    # Hàm và struct cho Producer
│       └── consumer.go    # Hàm và struct cho Consumer
├── config/                 # Cấu hình ứng dụng
│   └── config.go          # Load config từ biến môi trường hoặc file
└── models/                 # Định nghĩa các struct dữ liệu
    └── order.go           # Ví dụ: struct Order
```

4.2 Giải thích vai trò từng thư mục

Thư mục / File	Vai trò và lý do
cmd/	Chứa các chương trình thực thi. Mỗi sub-folder là một binary riêng. Go convention: main package phải ở đây.
internal/	Code chỉ dùng trong module này. Go compiler từ chối import internal/ từ module khác — tạo ra boundary rõ ràng.
config/	Tách biệt cấu hình ra khỏi logic. Dễ thay đổi địa chỉ Kafka mà không cần sửa code.
models/	Định nghĩa cấu trúc dữ liệu dùng chung. Tránh lặp code khi nhiều package cần cùng một struct.
go.mod	Khai báo tên module và phiên bản Go. Được tạo bởi lệnh go mod init. Tương đương package.json trong Node.js.

4.3 Khởi tạo project

```
# Tạo thư mục và vào đó
mkdir kafka-go-demo && cd kafka-go-demo

# Khởi tạo Go module
# Quy ước đặt tên module: github.com/<username>/<project>
```

```
go mod init github.com/hocsinh/kafka-go-demo

# Tạo cấu trúc thư mục
mkdir -p cmd/producer cmd/consumer internal/kafka config models

# Cài thư viện kafka-go
go get github.com/segmentio/kafka-go

# Kiểm tra go.mod — sẽ thấy kafka-go được thêm vào
cat go.mod
```

💡 Tại sao dùng github.com/segmentio/kafka-go?

Đây là thư viện Kafka phổ biến nhất cho Go, thuần Go (không dùng cgo).
API đơn giản, dễ hiểu hơn so với `confluent-kafka-go`.
Hỗ trợ đầy đủ Kafka 2.x và 3.x.
Tài liệu tốt, cộng đồng lớn, được maintain tích cực.

📋 Tóm tắt chương

Dùng thư mục `cmd/` cho các chương trình thực thi.
Dùng `internal/` cho logic nội bộ, `config/` cho cấu hình, `models/` cho struct.
Luôn chạy `go mod init` trước khi bắt đầu code.
Cài thư viện bằng `go get github.com/segmentio/kafka-go`.

Chương 5: Thực Hành — Kết Nối Go với Kafka

Đây là chương quan trọng nhất. Chúng ta sẽ viết code thực tế, chạy được ngay, và hiểu từng dòng một. Tất cả file trong chương này đều đặt theo cấu trúc thư mục đã thiết kế ở Chương 4.

5.1 Cấu hình kết nối — config/config.go

Đầu tiên, tạo file config để quản lý cấu hình tập trung. Không hardcode địa chỉ Kafka trong logic code!

```
// File: config/config.go
package config

import "os"

// KafkaConfig chứa toàn bộ cấu hình kết nối Kafka
type KafkaConfig struct {
    // Brokers là danh sách địa chỉ Kafka broker
    // Trong môi trường dev: chỉ cần 1 broker
    // Trong production: thường có 3+ broker để HA
    Brokers []string

    // Topic mặc định cho bài học này
    Topic string

    // GroupID dùng cho Consumer Group
    GroupID string
}

// LoadKafkaConfig đọc config từ biến môi trường.
// Nếu biến môi trường không được set, dùng giá trị mặc định.
func LoadKafkaConfig() KafkaConfig {
    broker := getEnv("KAFKA_BROKER", "localhost:9092")
    return KafkaConfig{
        Brokers: []string{broker},
        Topic:   getEnv("KAFKA_TOPIC", "don-hang"),
        GroupID: getEnv("KAFKA_GROUP_ID", "nhom-xu-ly-don-hang"),
    }
}

// getEnv là hàm tiện ích: trả về giá trị env hoặc defaultValue nếu chưa set
func getEnv(key, defaultValue string) string {
    if val := os.Getenv(key); val != "" {
        return val
    }
    return defaultValue
}
```

5.2 Model dữ liệu — models/order.go

Định nghĩa struct cho message mà chúng ta sẽ gửi qua Kafka. Trong ví dụ này, chúng ta mô phỏng hệ thống đặt hàng.

```
// File: models/order.go
package models

import "time"
```

```
// Order đại diện cho một đơn hàng trong hệ thống
type Order struct {
    // ID là mã đơn hàng duy nhất, dùng làm Kafka message key
    ID string `json:"id"`

    // CustomerID là mã khách hàng đặt đơn
    CustomerID string `json:"customer_id"`

    // Items là danh sách sản phẩm trong đơn hàng
    Items []OrderItem `json:"items"`

    // TotalAmount là tổng giá trị đơn hàng (VNĐ)
    TotalAmount float64 `json:"total_amount"`

    // Status: "pending", "confirmed", "shipped", "delivered"
    Status string `json:"status"`

    // CreatedAt là thời điểm đặt hàng
    CreatedAt time.Time `json:"created_at"`
}

// OrderItem đại diện cho một sản phẩm trong đơn hàng
type OrderItem struct {
    ProductID string `json:"product_id"`
    Name      string  `json:"name"`
    Quantity  int     `json:"quantity"`
    Price     float64 `json:"price"`
}
```

5.3 Lớp Kafka — internal/kafka/producer.go

Tạo một lớp bọc (wrapper) cho kafka-go để code ở tầng trên dễ sử dụng hơn và dễ test hơn.

```
// File: internal/kafka/producer.go
package kafka

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "github.com/segmentio/kafka-go"
)

// Producer bọc kafka.Writer và thêm logic tiện ích
type Producer struct {
    writer *kafka.Writer
}

// NewProducer tạo một Producer mới kết nối đến Kafka
// brokers: danh sách địa chỉ broker, ví dụ: []string{"localhost:9092"}
// topic: tên topic sẽ ghi message vào
func NewProducer(brokers []string, topic string) *Producer {
    writer := &kafka.Writer{
        // Addr là địa chỉ Kafka broker
        Addr: kafka.TCP(brokers...),
    }
    return &Producer{writer: writer}
```

```
// Topic là tên topic đích
Topic: topic,

// Balancer quyết định message đi vào Partition nào.
// LeastBytes: chọn partition có ít dữ liệu nhất (cân bằng tải)
Balancer: &kafka.LeastBytes{},

// WriteTimeout: nếu ghi không xong trong 10s thì trả lỗi
WriteTimeout: 10 * time.Second,

// ReadTimeout: timeout cho thao tác đọc metadata
ReadTimeout: 10 * time.Second,

// RequiredAcks: -1 (AllAcks) = chờ tất cả replica xác nhận.
// Đảm bảo message không bị mất nếu broker crash.
RequiredAcks: kafka.RequireAll,

// AllowAutoTopicCreation: tự động tạo topic nếu chưa tồn tại.
// Tiện trong môi trường dev, nên tắt trong production.
AllowAutoTopicCreation: true,
}

return &Producer{writer: writer}
}

// SendMessage gửi một object bất kỳ lên Kafka dưới dạng JSON.
// key: khóa của message (dùng để định tuyến vào partition)
// value: dữ liệu cần gửi (sẽ được chuyển sang JSON tự động)
func (p *Producer) SendMessage(ctx context.Context, key string, value any) error {
    // Chuyển value thành JSON bytes
    jsonBytes, err := json.Marshal(value)
    if err != nil {
        return fmt.Errorf("không thể marshal dữ liệu thành JSON: %w", err)
    }

    // Tạo message Kafka
    msg := kafka.Message{
        Key:    []byte(key),
        Value:  jsonBytes,
        // Time sẽ được gán tự động bởi Kafka
    }

    // Gửi message, nếu thất bại trả về lỗi có context
    if err := p.writer.WriteMessages(ctx, msg); err != nil {
        return fmt.Errorf("gửi message thất bại (key=%s): %w", key, err)
    }

    return nil
}

// Close đóng kết nối Writer. Luôn gọi hàm này khi không dùng nữa.
// Dùng defer producer.Close() sau khi tạo producer.
func (p *Producer) Close() error {
    return p.writer.Close()
}
```

5.4 Lớp Kafka — internal/kafka/consumer.go

```
// File: internal/kafka/consumer.go
```

```
package kafka

import (
    "context"
    "fmt"
    "time"

    "github.com/segmentio/kafka-go"
)

// Consumer bọc kafka.Reader
type Consumer struct {
    reader *kafka.Reader
}

// Message là struct đơn giản chứa thông tin message đã nhận
type Message struct {
    Key      string    // Khóa của message
    Value    []byte    // Nội dung message (JSON bytes)
    Partition int       // Partition nơi message nằm
    Offset   int64     // Offset của message trong partition
    Timestamp time.Time // Thời điểm Kafka nhận message
}

// NewConsumer tạo Consumer mới thuộc một Consumer Group.
// groupID: mỗi Consumer Group có tên riêng.
// Các Consumer có cùng groupID sẽ chia nhau các Partition.
func NewConsumer(brokers []string, topic, groupID string) *Consumer {
    reader := kafka.NewReader(kafka.ReaderConfig{
        Brokers: brokers,
        Topic:   topic,

        // GroupID định danh Consumer Group.
        // Kafka dùng GroupID để theo dõi offset đã commit.
        GroupID: groupID,

        // MinBytes và MaxBytes kiểm soát kích thước fetch.
        // 10KB - 10MB là giá trị hợp lý cho hầu hết trường hợp.
        MinBytes: 10e3, // 10KB
        MaxBytes: 10e6, // 10MB

        // MaxWait: nếu chưa đủ MinBytes sau thời gian này, trả về ngay.
        // Giảm độ trễ khi lượng message ít.
        MaxWait: 1 * time.Second,

        // CommitInterval: tự động commit offset mỗi 1 giây.
        // Trong production, hãy cân nhắc commit thủ công để kiểm soát tốt hơn.
        CommitInterval: 1 * time.Second,
    })

    return &Consumer{reader: reader}
}

// ReadMessage đọc message tiếp theo từ Kafka.
// Hàm này sẽ BLOCK (chờ) nếu chưa có message mới.
// Truyền ctx với timeout để tránh chờ mãi mãi.
func (c *Consumer) ReadMessage(ctx context.Context) (Message, error) {
    msg, err := c.reader.ReadMessage(ctx)
    if err != nil {
        return Message{}, fmt.Errorf("đọc message thất bại: %w", err)
    }

    return Message{
```

```

        Key:      string(msg.Key),
        Value:    msg.Value,
        Partition: msg.Partition,
        Offset:   msg.Offset,
        Timestamp: msg.Time,
    }, nil
}

// Close đóng kết nối Reader.
func (c *Consumer) Close() error {
    return c.reader.Close()
}

```

5.5 Producer — cmd/producer/main.go

Đây là chương trình Producer hoàn chỉnh. Nó sẽ giả lập việc tạo 5 đơn hàng và gửi lên Kafka.

```

// File: cmd/producer/main.go
package main

import (
    "context"
    "fmt"
    "log"
    "time"

    "github.com/hocsinh/kafka-go-demo/config"
    kafkaclient "github.com/hocsinh/kafka-go-demo/internal/kafka"
    "github.com/hocsinh/kafka-go-demo/models"
)

func main() {
    // Bước 1: Load cấu hình
    cfg := config.LoadKafkaConfig()
    log.Printf("Kết nối đến Kafka: %v, Topic: %s\n", cfg.Brokers, cfg.Topic)

    // Bước 2: Tạo Producer
    producer := kafkaclient.NewProducer(cfg.Brokers, cfg.Topic)
    // defer đảm bảo Close() được gọi khi hàm main kết thúc
    defer func() {
        if err := producer.Close(); err != nil {
            log.Printf("Lỗi khi đóng producer: %v\n", err)
        }
    }()

    // Bước 3: Giả lập tạo 5 đơn hàng và gửi lên Kafka
    orders := createSampleOrders()

    for _, order := range orders {
        // Context với timeout 5 giây cho mỗi lần gửi
        ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)

        // Gửi đơn hàng lên Kafka
        // Key = order.ID: đảm bảo cùng đơn hàng vào cùng partition
        err := producer.SendMessage(ctx, order.ID, order)
        cancel() // Luôn gọi cancel để giải phóng tài nguyên

        if err != nil {
            log.Printf("Gửi thất bại (ID=%s): %v\n", order.ID, err)
        }
    }
}

```

```

        continue
    }

    log.Printf("Đã gửi đơn hàng: ID=%s, Khách=%s, Tổng=%.0f VNĐ\n",
        order.ID, order.CustomerID, order.TotalAmount)

    // Dừng 500ms giữa các lần gửi để dễ quan sát
    time.Sleep(500 * time.Millisecond)
}

log.Println("Hoàn tất! Đã gửi tất cả đơn hàng lên Kafka.")
}

// createSampleOrders tạo dữ liệu mẫu để test
func createSampleOrders() []models.Order {
    return []models.Order{
        {
            ID:          "ORD-001",
            CustomerID: "KH-42",
            Status:    "pending",
            CreatedAt: time.Now(),
            TotalAmount: 350000,
            Items: []models.OrderItem{
                {ProductID: "SP-001", Name: "Cà phê phin", Quantity: 2, Price:
45000},
                {ProductID: "SP-002", Name: "Bánh mì thịt", Quantity: 3, Price:
25000},
                {ProductID: "SP-003", Name: "Nước cam", Quantity: 1, Price:
35000},
            },
        },
        {
            ID:          "ORD-002",
            CustomerID: "KH-15",
            Status:    "pending",
            CreatedAt: time.Now(),
            TotalAmount: 199000,
            Items: []models.OrderItem{
                {ProductID: "SP-004", Name: "Trà sữa trân châu", Quantity: 2,
Price: 55000},
                {ProductID: "SP-005", Name: "Bánh flan", Quantity: 1, Price:
25000},
            },
        },
        {
            ID:          "ORD-003",
            CustomerID: "KH-08",
            Status:    "pending",
            CreatedAt: time.Now(),
            TotalAmount: 520000,
            Items: []models.OrderItem{
                {ProductID: "SP-006", Name: "Bún bò Huế", Quantity: 4, Price:
65000},
            },
        },
    }
}

```

5.6 Consumer — cmd/consumer/main.go

Chương trình Consumer sẽ lắng nghe liên tục và in ra các đơn hàng nhận được từ Kafka.

```
// File: cmd/consumer/main.go
package main

import (
    "context"
    "encoding/json"
    "errors"
    "log"
    "os"
    "os/signal"
    "syscall"

    "github.com/hocsinh/kafka-go-demo/config"
    kafkaclient "github.com/hocsinh/kafka-go-demo/internal/kafka"
    "github.com/hocsinh/kafka-go-demo/models"
)

func main() {
    cfg := config.LoadKafkaConfig()
    log.Printf("Consumer khởi động - Topic: %s, Group: %s\n",
        cfg.Topic, cfg.GroupID)

    // Tạo Consumer thuộc group "nhom-xu-ly-don-hang"
    consumer := kafkaclient.NewConsumer(cfg.Brokers, cfg.Topic, cfg.GroupID)
    defer func() {
        if err := consumer.Close(); err != nil {
            log.Printf("Lỗi khi đóng consumer: %v\n", err)
        }
    }()

    // Thiết lập xử lý tín hiệu Ctrl+C để dừng gracefully
    // Graceful shutdown = dừng lại sạch sẽ, không bỏ dở message đang xử lý
    ctx, cancel := signal.NotifyContext(context.Background(),
        os.Interrupt, syscall.SIGTERM)
    defer cancel()

    log.Println("Đang lắng nghe message... (Nhấn Ctrl+C để dừng)")

    // Vòng lặp vô hạn để đọc message liên tục
    for {
        // ReadMessage sẽ block cho đến khi có message mới
        // hoặc ctx bị cancel (Ctrl+C)
        msg, err := consumer.ReadMessage(ctx)

        if err != nil {
            // errors.Is kiểm tra xem lỗi có phải do context bị cancel không
            if errors.Is(ctx.Err(), context.Canceled) {
                log.Println("Nhận tín hiệu dừng, thoát chương trình...")
                break
            }
            log.Printf("Lỗi đọc message: %v\n", err)
            continue
        }

        // Xử lý message nhận được
        processOrder(msg)
    }

    log.Println("Consumer đã dừng hoàn toàn.")
}

// processOrder xử lý một đơn hàng nhận từ Kafka
func processOrder(msg kafkaclient.Message) {
    // Parse JSON bytes thành struct Order
}
```

```
var order models.Order
if err := json.Unmarshal(msg.Value, &order); err != nil {
    log.Printf("Lỗi parse JSON (offset=%d): %v\n", msg.Offset, err)
    return
}

// In thông tin đơn hàng ra console
log.Printf("Nhận đơn hàng mới:")
log.Printf("  Partition: %d | Offset: %d", msg.Partition, msg.Offset)
log.Printf("  ID đơn hàng: %s", order.ID)
log.Printf("  Khách hàng: %s", order.CustomerID)
log.Printf("  Tổng tiền: %.0f VNĐ", order.TotalAmount)
log.Printf("  Số món: %d sản phẩm", len(order.Items))
log.Printf("  Trạng thái: %s", order.Status)
log.Println("---")

// Trong ứng dụng thực tế, đây là nơi bạn:
// - Lưu vào database
// - Gửi email xác nhận
// - Cập nhật kho hàng
// - v.v...
}
```

5.7 Chạy thử và kiểm tra kết quả

Mở hai cửa sổ terminal (terminal A và terminal B) và làm theo thứ tự:

Bước 1: Đảm bảo Kafka đang chạy

```
# Kiểm tra trạng thái
docker compose ps

# Nếu chưa chạy, khởi động lại
docker compose up -d

# Chờ Kafka healthy (có thể mất 30 giây)
docker compose ps # Cột STATUS phải là "Up (healthy)"
```

Bước 2: Chạy Consumer (Terminal A)

```
# Terminal A – Chạy Consumer TRƯỚC
cd kafka-go-demo
go run ./cmd/consumer/

# Output mong đợi:
# 2025/04/01 10:00:00 Consumer khởi động – Topic: don-hang, Group: nhom-xu-ly-don-hang
# 2025/04/01 10:00:00 Đang lắng nghe message... (Nhấn Ctrl+C để dừng)
# (Chương trình block tại đây, chờ message)
```

Bước 3: Chạy Producer (Terminal B)

```
# Terminal B – Chạy Producer
cd kafka-go-demo
go run ./cmd/producer/
```

```
# Output mong đợi:
# 2025/04/01 10:00:05 Kết nối đến Kafka: [localhost:9092], Topic: don-hang
# 2025/04/01 10:00:05 Đã gửi đơn hàng: ID=ORD-001, Khách=KH-42, Tổng=350000 VNĐ
# 2025/04/01 10:00:06 Đã gửi đơn hàng: ID=ORD-002, Khách=KH-15, Tổng=199000 VNĐ
# 2025/04/01 10:00:06 Đã gửi đơn hàng: ID=ORD-003, Khách=KH-08, Tổng=520000 VNĐ
# 2025/04/01 10:00:07 Hoàn tất! Đã gửi tất cả đơn hàng lên Kafka.
```

Bước 4: Quan sát Consumer nhận message (Terminal A)

```
# Ngay sau khi Producer gửi, Terminal A sẽ hiện:

# 2025/04/01 10:00:05 Nhận đơn hàng mới:
# 2025/04/01 10:00:05   Partition: 0 | Offset: 0
# 2025/04/01 10:00:05   ID đơn hàng: ORD-001
# 2025/04/01 10:00:05   Khách hàng:   KH-42
# 2025/04/01 10:00:05   Tổng tiền:    350000 VNĐ
# 2025/04/01 10:00:05   Số món:       3 sản phẩm
# 2025/04/01 10:00:05   Trạng thái:   pending
# 2025/04/01 10:00:05 ---
```

Bước 5: Kiểm tra qua Redpanda Console

Mở trình duyệt, truy cập <http://localhost:9091>. Bạn sẽ thấy:

- Topic "don-hang" đã được tạo tự động
- Số message trong topic (Message Count)
- Các Partition và offset
- Có thể click vào từng message để xem nội dung JSON

Tóm tắt chương

Config được tập trung tại `config/config.go`, dễ thay đổi môi trường.

Producer dùng `kafka.Writer`, Consumer dùng `kafka.Reader` từ thư viện `kafka-go`.

Luôn dùng context với timeout để tránh hang vô hạn.

`defer Close()` để đảm bảo đóng kết nối khi chương trình kết thúc.

Consumer đọc liên tục trong vòng lặp `for`, graceful shutdown bằng `signal`.

Kiểm tra kết quả qua Redpanda Console tại <http://localhost:9091>.

Chương 6: Lỗi Thường Gặp và Cách Xử Lý

Phần này tổng hợp những lỗi phổ biến nhất mà sinh viên gặp khi làm việc với Go và Kafka lần đầu. Mỗi lỗi đều kèm theo cách chẩn đoán và sửa chữa cụ thể.

6.1 Lỗi kết nối Kafka

```
# Lỗi: dial tcp [::1]:9092: connect: connection refused
# Hoặc: kafka: client has run out of available brokers
```

Nguyên nhân: Kafka chưa chạy, hoặc địa chỉ broker bị sai.

Cách xử lý:

1. Kiểm tra Kafka đang chạy: `docker compose ps` — cột STATUS phải là "Up (healthy)"
2. Nếu chưa chạy: `docker compose up -d`, đợi 30 giây
3. Nếu đang chạy mà vẫn lỗi: kiểm tra xem bạn có đang dùng đúng cổng không (mặc định 9092)
4. Test kết nối thủ công: `nc -zv localhost 9092`

⚠ Lỗi hay gặp: sai địa chỉ broker

Trong docker-compose, `KAFKA_ADVERTISED_LISTENERS` dùng `localhost:9092` cho client bên ngoài.

Nếu bạn chạy Go app TRONG Docker, phải dùng `kafka:19092` (tên service nội bộ).

Nếu bạn chạy Go app NGOÀI Docker (trực tiếp trên máy), dùng `localhost:9092`.

6.2 Lỗi topic chưa tồn tại

```
# Lỗi: kafka: topic authorization failed
# Hoặc: [5] Leader Not Available: the cluster is in the middle of a leadership election
```

Nguyên nhân: Topic chưa được tạo, Kafka đang khởi động, hoặc Kafka chưa ổn định.

Cách xử lý:

5. Đợi thêm 30-60 giây sau khi khởi động Kafka
6. Tạo topic thủ công nếu cần:

```
# Tạo topic thủ công với 3 partition
docker exec my-kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create \
  --topic don-hang \
  --partitions 3 \
  --replication-factor 1

# Liệt kê tất cả topic
docker exec my-kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 --list
```

6.3 Lỗi biên dịch Go

```
# Lỗi: cannot find package "github.com/segmentio/kafka-go"
# Hoặc: go: module lookup disabled by GONOSUMCHECK
```

Cách xử lý:

- Đảm bảo đã chạy `go mod init` trước
- Chạy `go get github.com/segmentio/kafka-go` để tải thư viện
- Nếu vẫn lỗi, chạy `go mod tidy` để dọn dẹp dependencies

```
# Sửa lỗi module
go mod tidy
go mod download

# Kiểm tra go.mod đã có kafka-go
cat go.mod | grep kafka
```

6.4 Consumer không nhận được message

Nguyên nhân thường gặp:

- Consumer khởi động SAU khi Producer gửi: theo mặc định, Consumer Group mới bắt đầu đọc từ offset mới nhất (latest), không đọc lại message cũ.
- Consumer Group ID bị trùng với group khác.
- Topic bị viết sai tên.

Cách xử lý:

- Luôn chạy Consumer TRƯỚC, rồi mới chạy Producer.
- Để đọc lại message cũ từ đầu, thêm `StartOffset`:

```
// Trong NewConsumer, thêm StartOffset để đọc từ đầu topic
reader := kafka.NewReader(kafka.ReaderConfig{
    // ... các cấu hình khác ...

    // StartOffset: kafka.FirstOffset = đọc từ message đầu tiên
    // Chỉ áp dụng nếu GroupID chưa có commit offset nào trước đó
    StartOffset: kafka.FirstOffset,
})
```

6.5 Lỗi context deadline exceeded

```
# Lỗi: context deadline exceeded
# Hoặc: write tcp [::1]:xxxxx->[::1]:9092: i/o timeout
```

Nguyên nhân: Timeout quá ngắn, hoặc Kafka đang quá tải.

Cách xử lý:

- Tăng timeout trong context: `context.WithTimeout(ctx, 30*time.Second)`
- Kiểm tra tài nguyên máy: `docker stats my-kafka`
- Xem log Kafka: `docker compose logs --tail=50 kafka`

6.6 Bảng tổng hợp lỗi nhanh

Thông báo lỗi	Nguyên nhân	Giải pháp nhanh
connection refused	Kafka chưa chạy	docker compose up -d
cannot find package	Chưa cài thư viện	go get kafka-go
Leader Not Available	Kafka đang khởi động	Chờ thêm 30 giây
context deadline exceeded	Timeout quá ngắn	Tăng timeout lên 30s
Consumer không nhận msg	Chạy Consumer sau Producer	Chạy Consumer trước
duplicate consumer group	Trùng GroupID	Đổi tên GroupID

 Tóm tắt chương

- Luôn kiểm tra docker compose ps trước khi chạy Go code.
- Đọc kỹ thông báo lỗi — Kafka và kafka-go thường có message rất rõ ràng.
- Dùng docker compose logs kafka để xem log phía server.
- Consumer phải khởi động TRƯỚC Producer để không bỏ sót message.
- Tăng timeout nếu gặp deadline exceeded trong môi trường phát triển.

Chương 7: Tổng Kết

7.1 Những gì bạn đã học được

Qua tài liệu này, bạn đã đi một hành trình từ con số 0 đến một ứng dụng Go-Kafka hoàn chỉnh, hoạt động được. Hãy điểm lại những gì đã nắm được:

15. **Go là gì:** ngôn ngữ biên dịch, nhanh, concurrency tốt, được dùng rộng rãi cho backend và hệ thống phân tán
16. **Kafka là gì:** message broker phân tán, giúp các service giao tiếp qua mô hình publish-subscribe
17. **Môi trường Dev:** cài Go 1.22+, Docker, và chạy Kafka bằng docker compose up -d
18. **7 khái niệm Kafka:** Message, Topic, Partition, Offset, Broker, Producer, Consumer Group
19. **Cấu trúc project:** cmd/, internal/, config/, models/ — chuẩn Go, dễ mở rộng
20. **Code thực tế:** Producer gửi JSON message, Consumer đọc và xử lý, graceful shutdown
21. **Debug:** biết cách đọc lỗi, kiểm tra log, và xử lý tình huống thường gặp

7.2 Bảng ôn tập khái niệm

Khái niệm	Định nghĩa	Ví dụ thực tế
Message	Đơn vị dữ liệu (key + value)	{ "order_id": "ORD-001", ... }
Topic	Kênh phân loại message	"don-hang", "thanh-toan"
Partition	Phân vùng lưu trữ trong Topic	Topic 3 partition = 3 queue song song
Offset	Số thứ tự message trong Partition	Offset 0, 1, 2, 3... tăng dần mãi
Broker	Server Kafka	Container my-kafka ở cổng 9092
Producer	Ghi message vào Topic	App đặt hàng gửi event lên Kafka
Consumer	Đọc message từ Topic	App email đọc đơn hàng và gửi mail
Consumer Group	Nhóm Consumer cùng làm việc	3 instance email-service chia 3 partition
Retention	Thời gian giữ message	Mặc định 7 ngày, Consumer đọc lại được

7.3 Lộ trình học tiếp theo

7.3.1 Chủ đề nâng cao về Go

- **Goroutine và Channel:** nền tảng concurrency của Go — đây là lý do Go được chọn cho hệ thống phân tán
- **context.Context:** quản lý timeout, cancellation, và truyền metadata xuyên suốt call stack
- **Error handling nâng cao:** dùng errors.Is, errors.As, và custom error types

- **Testing:** viết unit test với testing package và mock Kafka connection
- **Middleware và interceptor:** thêm logging, metrics, tracing vào ứng dụng
- **Dependency Injection:** dùng wire hoặc fx để quản lý dependencies

7.3.2 Chủ đề nâng cao về Kafka

- **Consumer Group nâng cao:** rebalancing, offset management thủ công, exactly-once semantics
- **Kafka Streams:** xử lý và biến đổi stream dữ liệu trong thời gian thực
- **Schema Registry:** quản lý schema Avro/Protobuf để đảm bảo tính nhất quán của message
- **Kafka Connect:** đồng bộ dữ liệu giữa Kafka và database/storage khác
- **Monitoring:** theo dõi Kafka với Prometheus và Grafana
- **Security:** cấu hình TLS, SASL authentication cho môi trường production

7.3.3 Tài nguyên học thêm

Loại	Tên tài nguyên	Mô tả
Tài liệu chính thức	go.dev/doc/	Tài liệu Go đầy đủ và chính xác nhất
Tài liệu chính thức	kafka.apache.org/documentation	Tài liệu Kafka từ Apache
Sách	The Go Programming Language	Donovan & Kernighan — sách Go tốt nhất
Sách	Kafka: The Definitive Guide	Sách Kafka từ các kỹ sư Confluent
Khóa học	go.dev/tour/	Tour of Go — học Go tương tác, miễn phí
Thực hành	github.com/segmentio/kafka-go	Docs và ví dụ của thư viện kafka-go
Cộng đồng	gophers.slack.com	Slack của cộng đồng Go toàn cầu

📌 Lời khuyên từ giảng viên

Đừng cố học hết lý thuyết rồi mới code — hãy code ngay và học qua lỗi. Thay đổi code ví dụ, phá vỡ nó, và tìm hiểu tại sao nó hỏng. Đó là cách học tốt nhất. Xây dựng một mini-project thực tế (ví dụ: hệ thống chat đơn giản) để áp dụng kiến thức. Đọc code của người khác trên GitHub — đặc biệt là các project dùng kafka-go trong thực tế. Kafka và Go đều có documentation tốt bằng tiếng Anh. Đây là cơ hội tốt để rèn tiếng Anh kỹ thuật.

— Hết tài liệu —

Chúc bạn học tốt và code vui! 🚀